

Part 5, CI/CD Implementation Report

GitHub Actions, Playwright API automation and GitHub Pages reporting

Candidate	Jeremiah Igbinedion
Status	Completed, deployed and publicly accessible
Live dashboard	https://dionjerry.github.io/quality-engineering-project/
Workflow	<code>.github/workflows/api-tests.yml</code>

Overview

Part 5 implements a complete GitHub Actions pipeline for the Part 2 Playwright API suite. It performs static quality checks, provisions isolated Restful Booker services, executes 101 tests across four parallel shards, merges reports, retains diagnostic evidence and publishes the latest successful main-branch report to GitHub Pages.

Final implementation status

- CI is green on the verified feature and develop runs.
- The implementation has been promoted to main.
- GitHub Pages deployment is enabled and the public dashboard is live.
- The public dashboard is available at <https://dionjerry.github.io/quality-engineering-project/>.
- Only successful main runs can replace the latest green public report.

1. Workflow Design

Triggers and quality gate

- Runs on every push, pull requests targeting develop or main, manual workflow dispatch and a nightly 02:00 UTC schedule, equivalent to 03:00 WAT.
- Concurrency control cancels obsolete runs for the same branch or pull request.
- The first job runs npm ci, ESLint and strict TypeScript checking. API shards start only after this job succeeds.

Restful Booker service

Each shard starts an independent localhost Restful Booker container using the digest-pinned image mwint eringham/restfulbooker@sha256:e4a04233731ceca2e6d99bca16689e0d4bc44631047ab351b6f11b5fa08fb418. Port 3001 and the /ping health check are used. The public Restful Booker service is never used by CI.

Credentials

- API_USERNAME and API_PASSWORD are stored as GitHub Actions repository secrets.
- The suite produces a clear configuration error if either value is missing.
- The localhost base URL is defined in the workflow because it is not sensitive.
- No authentication credentials are committed to the repository.

2. Parallel Execution and Known Bugs

Four-way sharding

Shard	Tests
1 of 4	26
2 of 4	25
3 of 4	25
4 of 4	25
Total	101

Playwright uses test-level distribution with one worker per shard. Each shard has its own container and test data. fail-fast: false allows the other shards to retain evidence if one fails.

Expected-failure policy

- Confirmed defects assert the intended behaviour and are marked with Playwright `test.fail()` plus a stable BUG-API identifier.
- A reproduced known defect is visible as an expected failure without keeping the pipeline permanently red.
- An unexpected pass fails the run, signalling that the defect may have been fixed and its annotation must be reviewed.
- Unmarked regressions fail normally. Characterisation tests and unresolved product questions are not marked as defects.

3. Reporting, Artifacts and Pages

Merged reporting

- Every shard uploads a blob report and shard diagnostics.
- A dedicated merge job downloads all blob reports and creates one combined Playwright HTML report and one JUnit XML report.
- The final report title is Restful Booker API Test Report.
- Combined reports and diagnostic artifacts are retained for 14 days.
- Artifact uploads use `if: always()` so evidence is retained after unexpected failures.

Public dashboard

The responsive GitHub Pages dashboard is live at <https://dionjerry.github.io/quality-engineering-project/>. It shows the latest successful report status, test totals, known-bug checks, execution duration, four-shard details, expected-failure policy, a link to the complete Playwright report and recent workflow history with an Actions fallback.

Public evidence controls

- Only successful main runs deploy to Pages.
- Failed runs cannot overwrite the latest green report.
- Pull requests and non-main branches cannot deploy.
- JUnit XML, traces, screenshots, archives, error-context files and other diagnostics remain private workflow artifacts and are excluded from the public bundle.

4. Security and Reliability Controls

- Read-only repository permissions by default and job-specific Pages permissions.
- GitHub Secrets for credentials and localhost-only API execution.
- Digest-pinned Docker image, lockfile-reproducible npm ci and Node.js 22.
- Job timeouts, concurrency cancellation and fail-fast: false for shards.
- Separate public report sanitisation and failure-artifact retention.
- No force push, squash, rebase or backdating of assessment commits.

Verified Actions evidence

Feature branch run: <https://github.com/dionjerry/quality-engineering-project/actions/runs/30702797269>, successful. Develop branch run:

<https://github.com/dionjerry/quality-engineering-project/actions/runs/30702882572>, successful.

Verified outcomes

- Quality gate passed.
- All four API shards passed.
- 101 tests executed with zero unexpected failures.
- Combined HTML and JUnit reports were uploaded.
- The combined JUnit summary recorded tests=101, failures=0, skipped=0 and errors=0.
- GitHub Pages is now live from the main-branch deployment.

5. Engineering Corrections

Uneven shards

File-level distribution produced 27, 53, 0 and 21 tests. I changed it to test-level distribution, producing 26, 25, 25 and 25 tests.

Hardcoded credentials

Duplicated demo credentials were removed from test code and defaults. The final implementation uses required environment variables locally and repository secrets in CI.

Per-shard reports

Four separate reports were replaced with blob inputs and one merged HTML and JUnit result for easier review.

Public failure evidence

The Pages bundle was sanitised so diagnostic files remain workflow artifacts and are not publicly exposed.

Deployment scope

Pages publishing was limited to successful main runs to prevent pull requests, branches or failed executions from replacing the reviewer-facing report.

6. Commit and Verification Record

Progressive commits

- 33355e0, chore(ci): prepare Playwright suite for sharded execution
- c82c0cf, ci: run API suite in parallel GitHub Actions shards
- 70410e8, ci: merge sharded reports and publish test dashboard
- 0e905b4, merge: add Part 5 CI reporting dashboard

Human verification

- Reviewed branch and commit boundaries, repository secrets, workflow YAML and artifact inventories.
- Executed the Playwright suite locally and checked ESLint and strict TypeScript output.
- Verified four-way distribution, local report merging and the combined JUnit count.
- Reviewed desktop and mobile dashboard layouts, workflow-history fallback behaviour and Pages security filtering.
- Confirmed the public GitHub Pages URL after deployment.

Conclusion

Part 5 is complete. The pipeline meets the required push and pull-request CI behaviour, runs the Part 2 API suite using isolated containers, parallelises execution, preserves reports and failure evidence, handles expected failures without permanent red CI, includes nightly execution and sharding, and publishes the latest successful main result through the live GitHub Pages dashboard.